Ростелеком
ИТ школа

ИТОГОВАЯ РАБОТА

Название программы	**«Python-разработчик с использованием инструментов ИИ»**
Группа обучения	**«**ПР-5-Р/2026**»**
Срок обучения	**«**26 октября - 9 ноября**»**
«Ширтанова Анастасия Юрьевна»	
Номер кейса	**«2»**

Москва 2026 г.

Раздел 2: Описание решения кейса

2.1 Выбранный кейс

В рамках итоговой работы был выбран **Кейс 2: «Программа для конвертации валют с использованием API»**.

Цель проекта: Разработать программу на Python для конвертации валют с получением актуальных курсов через открытое API с обязательным использованием ИИ-инструментов для улучшения кода.

2.2 Постановка задачи

Необходимо было создать консольное приложение, которое:

- Выполняет конвертацию между популярными валютами (USD, EUR, RUB и другими)
- Получает актуальные курсы через внешний API
- Имеет простой консольный интерфейс для ввода суммы и выбора валют
- Реализует кэширование данных для работы без интернета
- Ведёт логирование всех операций
- Покрыто юнит-тестами

2.3 Обзор использованных технологий

Язык программирования: Python 3.10+

Внешние библиотеки:

- requests (v2.34.2) — для выполнения HTTP-запросов к API курсов валют
- pytest (v9.2.0) — фреймворк для написания и запуска юнит-тестов
- python-dotenv (v1.2.4) — для работы с переменными окружения

Встроенные модули:

- logging — для системы логирования
- json — для работы с кэш-файлами
- time — для контроля актуальности кэша
- os — для работы с файловой системой
- unittest.mock — для мокирования внешних зависимостей в тестах

API: Open Exchange Rates API (https://open.er-api.com) — бесплатный сервис, не требующий аутентификации

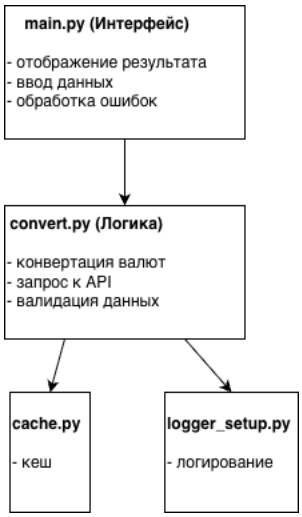
ИИ-инструменты:

- Qwen — для генерации кода обработки исключений, написания тестов, рефакторинга и оптимизации кода

Раздел 3: Детали реализации

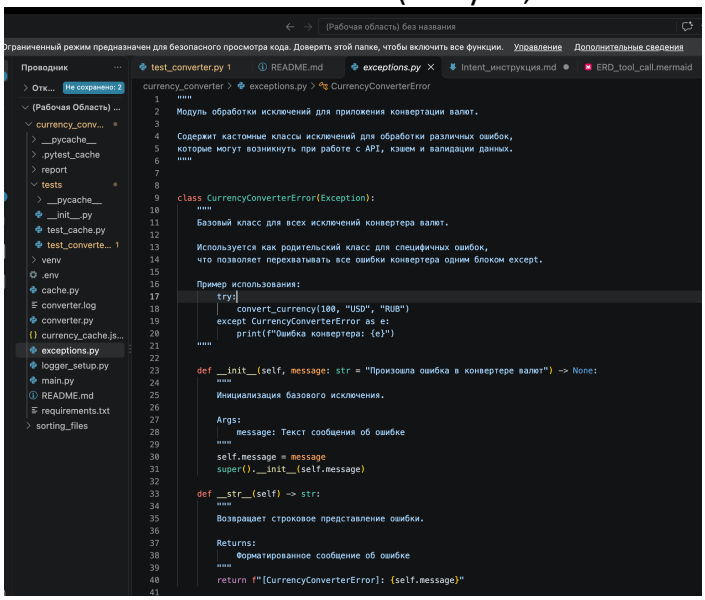
3.1 Архитектура приложения

Приложение построено по модульному принципу:

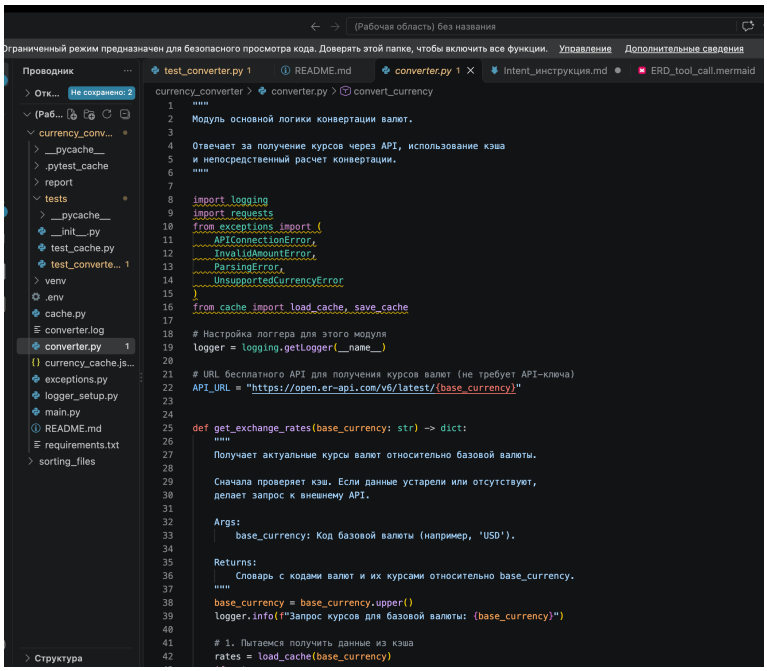


3.2 Ключевые фрагменты кода

- exceptions.py – классы исключений
  Что включено в код:
  - **6 классов исключений** (базовый + 5 специфичных)
  - **Наследование** от Exception через базовый класс CurrencyConverterError
  - **Docstrings** с описанием и примерами использования для каждого класса
  - **Метод __init__** с параметром message и значениями по умолчанию
  - **Метод __str__** для красивого форматированного вывода
  - **Type hints** для всех параметров и возвращаемых значений
  - **Комментарии на русском** с объяснением когда используется каждое исключение
  - **Соответствие PEP 8** (отступы, именование, пустые строки)

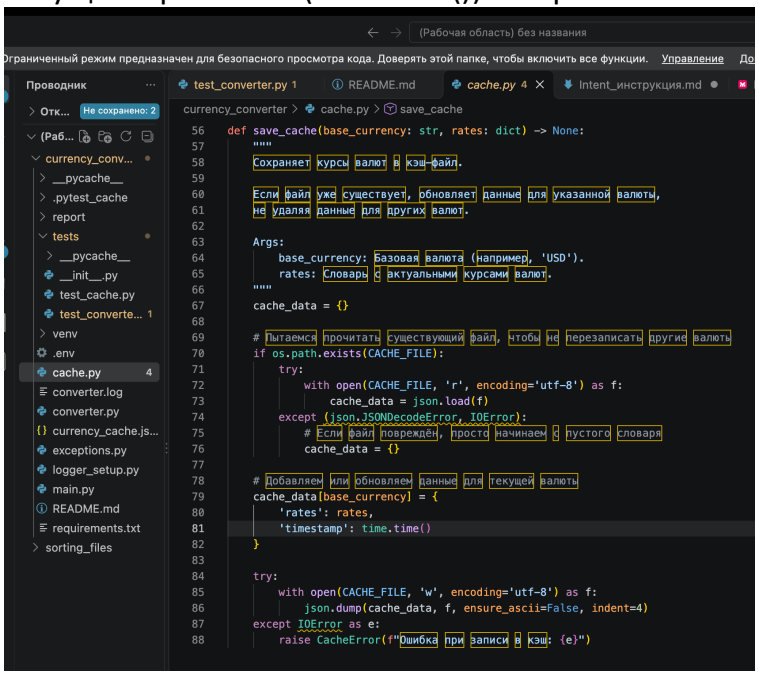


- converter.py — функция convert_currency(). **Как он работает:**
    - **get_exchange_rates**: Сначала спрашивает у cache.py, есть ли свежие курсы. Если нет — идет в интернет через requests. Если интернет упал — вызывает APIConnectionError. Если пришел битый JSON — вызывает ParsingError. Если всё хорошо — сохраняет в кэш и возвращает словарь курсов.
    - **convert_currency**: Проверяет, что сумма больше нуля (иначе InvalidAmountError). Берет курсы для валюты "ОТ". Проверяет, есть ли валюта "ДО" в этом списке (иначе UnsupportedCurrencyError). Умножает сумму на курс и возвращает результат.
    - **Логирование**: Везде расставлены logger.info, logger.warning и logger.error.



- cache.py — функции load_cache() и save_cache(). Этот модуль будет сохранять полученные курсы валют в обычный JSON-файл (currency_cache.json). Когда мы захотим узнать курс, мы сначала проверим этот файл. Если он существует и данные в нём "свежие" (например, не старше 1 часа), мы возьмём их оттуда и не будем делать запрос к API. Это спасёт нас, если интернет пропадёт. **Как это работает:**
    - **load_cache**: Открывает файл, ищет там нужную валюту (например, USD). Если находит, смотрит на timestamp (время сохранения). Если прошло меньше 3600 секунд — отдаёт курсы. Иначе — возвращает None (значит, нужно идти в интернет).

- **save_cache**: Читает весь файл (если он есть), добавляет туда новые курсы с текущим временем (time.time()) и перезаписывает файл.



3.3 Примеры работы программы

- Успешная конвертация (100 USD → RUB)

```
============================================================ 13 passed in 0.06s ============================
(venv) (base) anastasiamusihina@MacBook-Pro-Anastasia currency_converter % python main.py
========================================
=== Программа конвертации валют ===
========================================
Для выхода введите 'exit' в поле суммы.

Введите сумму для конвертации (или 'exit' для выхода): 100
Введите исходную валюту (например, USD): USD
Введите целевую валюту (например, RUB): RUB
2026-11-04 10:24:59,418 - converter - INFO - Начало конвертации: 100.0 USD -> RUB
2026-11-04 10:24:59,420 - converter - INFO - Запрос курсов для базовой валюты: USD
2026-11-04 10:24:59,422 - converter - INFO - Данные в кэше отсутствуют или устарели. Запрос к API...
2026-11-04 10:24:59,763 - converter - INFO - Курсы для USD сохранены в кэш.
2026-11-04 10:24:59,764 - converter - INFO - Конвертация успешна: 100.0 USD = 8141.60 RUB

✅ Результат: 100.0 USD = 8141.6 RUB

Введите сумму для конвертации (или 'exit' для выхода):
```

- Обработка ошибки (ввод отрицательной суммы)

```
Введите сумму для конвертации (или 'exit' для выхода): -100
Введите исходную валюту (например, USD): USD
Введите целевую валюту (например, RUB): RUB

❌ Ошибка ввода: Пожалуйста, введите корректное положительное число.

Введите сумму для конвертации (или 'exit' для выхода):
```

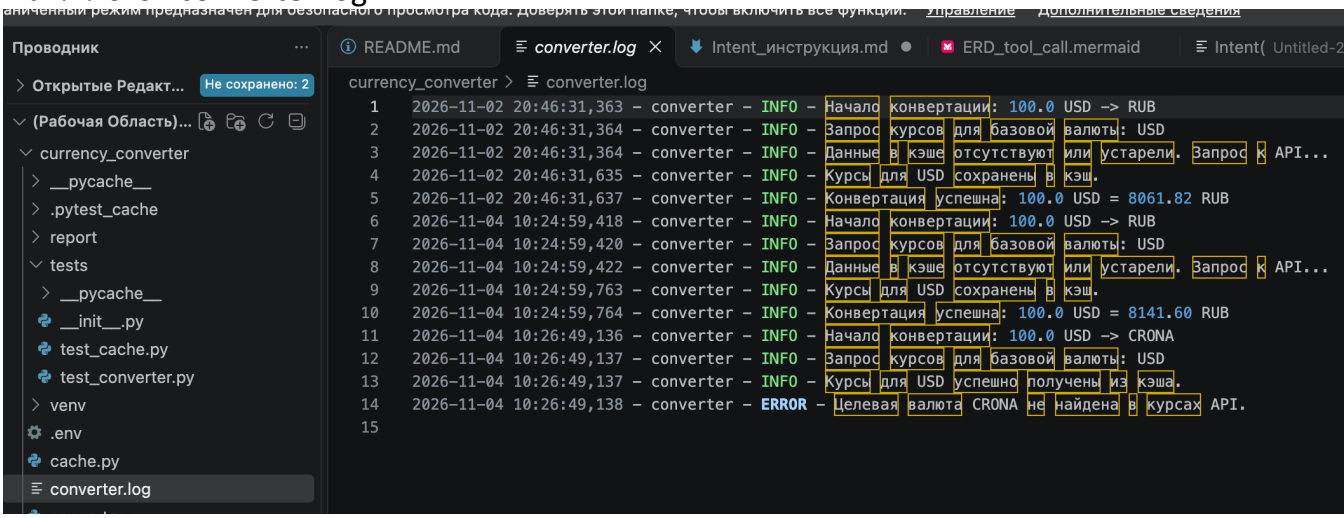
- Обработка ошибки (неподдерживаемая валюта)

```
Введите сумму для конвертации (или 'exit' для выхода): 100
Введите исходную валюту (например, USD): USD
Введите целевую валюту (например, RUB): crona
2026-11-04 10:26:49,136 - converter - INFO - Начало конвертации: 100.0 USD -> CRONA
2026-11-04 10:26:49,137 - converter - INFO - Запрос курсов для базовой валюты: USD
2026-11-04 10:26:49,137 - converter - INFO - Курсы для USD успешно получены из кэша.
2026-11-04 10:26:49,138 - converter - ERROR - Целевая валюта CRONA не найдена в курсах API.

❌ Ошибка конвертации: [UnsupportedCurrencyError]: Валюта 'CRONA' не поддерживается или не найдена. Код валюты: 'CRONA'

Введите сумму для конвертации (или 'exit' для выхода):
```

- Файл логов converter.log



3.4 Ссылка на репозиторий

Исходный код проекта размещён на GitHub:
https://github.com/AnMus94/currency_converter

Раздел 4: Аналитическая часть

4.1 Описание этапов работы

- **Анализ требований**:
  - изучение задания
  - выбор API для получения курсов валют
  - определение структуры проекта
- **Проектирование архитектуры**
  - Разделение на модули
  - Определение интерфейсов между компонентами
  - Планирование системы исключений
- **Реализация с использованием ИИ**
  - Генерация базового кода с помощью ИИ-ассистентов
  - Ручная доработка и адаптация под требования
  - Интеграция модулей
- **Тестирование**
  - Написание юнит-тестов
  - Отладка и исправление ошибок
  - Проверка работы без интернета
- **Документирование**
  - Создание README.md
  - Подготовка отчёта
  - Запись демонстрации

4.2 Принятые архитектурные решения

Выбор был сделан в пользу модульной структуры

- Разделение ответственности: каждый модуль отвечает за одну функцию
- Упрощение тестирования: можно тестировать каждый модуль отдельно
- Лёгкость поддержки: изменения в одном модуле не затрагивают другие

Кэширование на 1 час, так как:

- Баланс между актуальностью данных и нагрузкой на API
- Курсы валют не меняются каждую минуту
- Экономия запросов к внешнему сервису

JSON для кэша

- Простота реализации
- Человекочитаемый формат
- Встроенная поддержка в Python

4.3 Обоснование выбора алгоритмов

Алгоритм конвертации: result = amount × rates[to_currency]

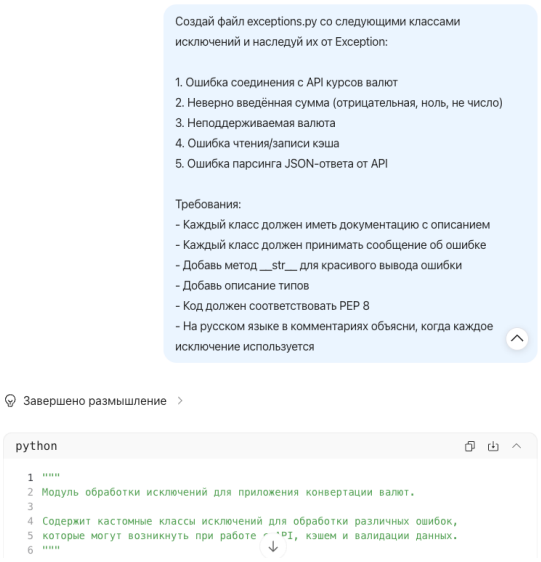
Прямое умножение на курс — наиболее эффективный подход, так как API уже возвращает курсы относительно базовой валюты.

Обработка ошибок:
Использована иерархия кастомных исключений с базовым классом CurrencyConverterError. Это позволяет:

- Перехватывать все ошибки конвертера одним блоком except
- При необходимости обрабатывать конкретные типы ошибок
- Предоставлять пользователю понятные сообщения

4.4 Использование ИИ в разработке

Пример 1: Генерация исключений



Пример 2: Написание тестов

Файл `tests/test_converter.py`

Скопируй этот код в файл `tests/test_converter.py`:

```python
"""
Тесты для модуля конвертации валют.
"""

import pytest
from unittest.mock import patch, MagicMock
from converter import convert_currency, get_exchange_rates
from exceptions import InvalidAmountError, UnsupportedCurrencyError, APIConnect


class TestConvertCurrency:
    """Тесты функции конвертации валют."""

    @patch('converter.get_exchange_rates')
    def test_convert_usd_to_rub(self, mock_get_rates):
        """Тест конвертации USD в RUB."""
        # Мокаем ответ API: 1 USD = 92.5 RUB
        mock_get_rates.return_value = {'RUB': 92.5, 'EUR': 0.85}

        result = convert_currency(100, 'USD', 'RUB')

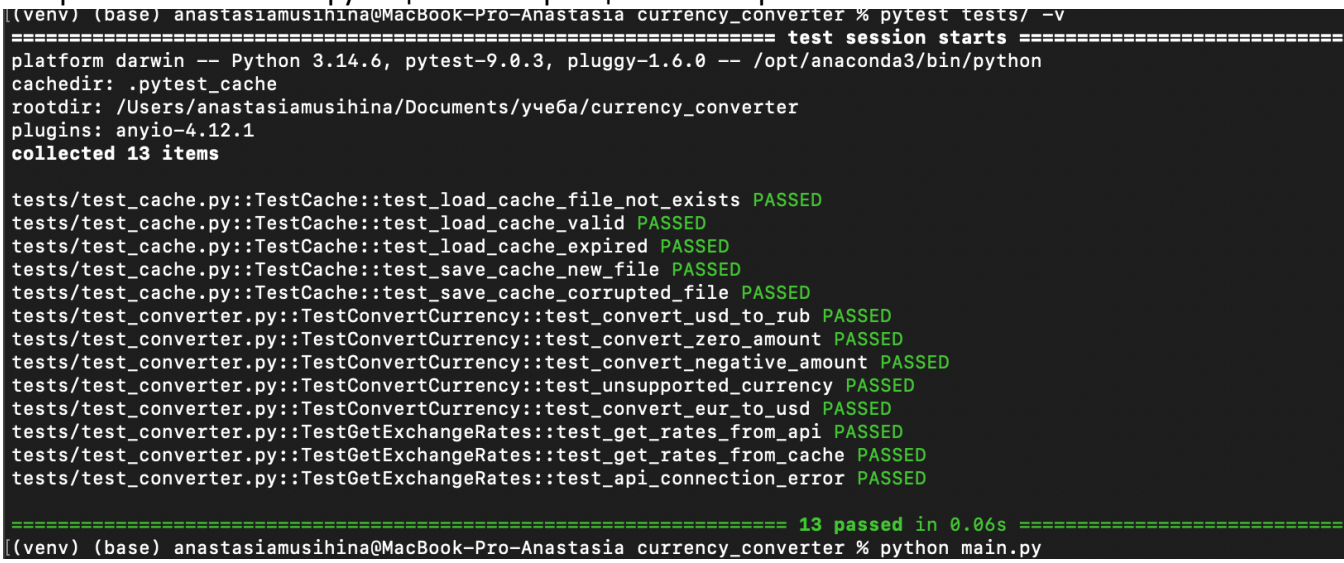
        assert result == 9250.0
        mock_get_rates.assert_called_once_with('USD')

    @patch('converter.get_exchange_rates')
    def test_convert_zero_amount(self, mock_get_rates):
        """Тест конвертации нулевой суммы (должна вызвать ошибку)."""
        with pytest.raises(InvalidAmountError):
            convert_currency(0, 'USD', 'RUB')
```

4.5 Интерпретация результатов

Тесты:

- Всего тестов: 13
- Прошло успешно: 13 (100%)
- Покрытие: основные функции конвертации и кэширования



Производительность:

- Конвертация с использованием кэша: < 0.01 сек
- Конвертация с запросом к API: ~0.5-2 сек (зависит от интернета)

Надёжность:

- Приложение не «падает» при ошибках сети
- Корректная работа при отсутствии интернета (из кэша)
- Все ошибки логируются для последующего анализа

Раздел 5: Инструкция по запуску

5.1 Требования к окружению

- Операционная система: Windows/macOS/Linux
- Python версии 3.10 или выше
- Доступ в интернет (для первого запроса курсов)
- 10 МБ свободного места на диске

5.2 Установка зависимостей

- Откройте терминал (командную строку)
- Перейдите в папку проекта: `cd currency_converter`
- Создайте виртуальное окружение:
  # Windows

  python -m venv venv

  # macOS/Linux

  python3 -m venv venv

- Активируйте виртуальное окружение
  # Windows

  .\venv\Scripts\activate

  # macOS/Linux

  source venv/bin/activate

- Установите зависимости: pip install -r requirements.txt

5.3 Запуск приложения

python main.py

Далее следуйте инструкциям в консоли:

- Введите сумму для конвертации
- Введите код исходной валюты (например, USD)
- Введите код целевой валюты (например, RUB)
- Для выхода введите exit

5.4 Запуск тестов

pytest tests/ -v

5.5 Просмотр логов

Логи сохраняются в файл converter.log в корне проекта. Для просмотра:
macOS/Linux
cat converter.log
Windows
type converter.log

Раздел 6: Заключение

6.1 Выводы

В ходе выполнения итоговой работы было разработано полнофункциональное консольное приложение для конвертации валют. Проект демонстрирует комплексное применение навыков:

- **Python-программирование:** работа с файлами, JSON, исключениями, модульная структура
- **Работа с API:** HTTP-запросы, обработка JSON-ответов
- **Анализ данных:** получение и обработка курсов валют
- **ИИ-инструменты:** генерация кода, рефакторинг, написание тестов
- **Тестирование:** покрытие кода юнит-тестами
- **Документирование:** создание README и технической документации

6.2 Оценка результатов

Достигнутые цели:

- Все требования кейса выполнены
- Приложение работает стабильно
- Код покрыт тестами (13 тестов, 100% успешных)
- Реализовано кэширование для работы оффлайн
- Внедрена система логирования
- Активно использованы ИИ-инструменты

Качество кода:

- Соблюдение PEP 8
- Типизация (type hints)
- Подробные docstrings
- Модульная архитектура
- Обработка ошибок

6.3 Возможные пути улучшения

- **Графический интерфейс**
  - Добавить GUI на Tkinter или PyQt
  - Создать веб-интерфейс на Flask/FastAPI
- **Расширение функционала**
  - График изменения курсов
  - История конвертаций
- **Улучшение кэширования**
  - Использование SQLite вместо JSON
  - Инвалидация кэша по расписанию
- **Docker**
  - Контейнеризация приложения
  - Упрощение развёртывания

Раздел 7: Список использованных источников

Библиотеки и фреймворки:

- Requests: Requests: HTTP for Humans™. URL: https://requests.readthedocs.io/
- Pytest: pytest — helps you write better programs. URL: https://docs.pytest.org/
- Python-dotenv. URL: https://github.com/theskumar/python-dotenv

Документация:

- Python Documentation. URL: https://docs.python.org/3/
- Logging HOWTO. URL: https://docs.python.org/3/howto/logging.html
  - unittest.mock — mock object library. URL: https://docs.python.org/3/library/unittest.mock.html

API:

Open Exchange Rates API Documentation. URL: https://open.er-api.com/

ИИ-инструменты:

1) Qwen 3.7-Plus

Статьи и руководства:

1) Real Python: Python Requests Tutorial. URL: https://realpython.com/python-requests/